

```
test-pc:~$python3
Python 3.6.7 (default, Oct 22 2018, 11:32:17)
[GCC 8.2.0] on linux
Type "help", "copyright", "credits" or "license" for more
information.
>>> import tensorflow as tf
>>> print(tf.__version__)
2.0.0-beta1
>>> tf.add(2,3)
<tf.Tensor: id=2, shape=(), dtype=int32, numpy=5>
```

In v1.x, the resulting Tensor doesn't display the value and we need to execute the graph under a session to get the value, but in v2.0 the values are implicitly computed and available for debugging.

Keras

Keras (*tf.keras*) is now the official high level API. It has been enhanced with many compatible low level APIs. The redundancy across Keras and TensorFlow is removed, and most of the APIs are now available with Keras. The low level operators are still accessible through *tf.raw_ops*.

We can now save the Keras model directly as a Tensorflow SavedModel, as shown below:

```
# Save Model to SavedModel
saved_model_path = tf.keras.experimental.export_saved_
model(model, '/path/to/model')

# Load the SavedModel
new_model = tf.keras.experimental.load_from_saved_
model(saved_model_path)

# new_model is now keras Model object.
new_model.summary()
```

Earlier, APIs related to various layers, optimisers, metrics and loss functions were distributed across Keras and native TensorFlow. Latest enhancements unify them as *tf.keras.optimizer.**, *tf.keras.metrics.**, *tf.keras.losses.** and *tf.keras.layers.**.

The RNN layers are now much more simplified compared to v 1.x.

With TensorFlow 1.x, the commands given are:

```
if tf.test.is_gpu_available():
    model.add(tf.keras.layers.CudnnLSTM(32))
else
    model.add(tf.keras.layers.LSTM(32))
```

With TensorFlow 2.0, the commands given are:

```
# This will use Cudnn kernel when the GPU is available.
```

```
model.add(tf.keras.layers.LSTM(32))
```

TensorBoard integration is now a simple call back, as shown below:

```
tb_callbackk = tf.keras.callbacks.TensorBoard(log_dir=log_
dir)

model.fit(
    x_train, y_train, epochs=5,
    validation_data = [x_test, y_test],
    Callbacks = [tb_callbacks])
```

With this simple call back addition, TensorBoard is up on the browser to look for all the statistics in real-time.

Keras offers unified distribution strategies, and a few lines of code can enable the required strategy as shown below:

```
strategy = tf.distribute.MirroredStrategy()

with strategy.scope()
    model = tf.keras.models.Sequential([
        tf.keras.layers.Dense(64, input_shape=[10]),
        tf.keras.layers.Dense(64, activation='relu'),
        tf.keras.layers.Dense(10, activation='softmax')])

    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
```

As shown above, the model definition under the desired scope is all we need to apply the desired strategy. Very soon, there will be support for multi-node synchronous and TPU strategy, and later, for parameter server strategy.

TensorFlow function

Function is a major upgrade that impacts the way we write TensorFlow applications. The new version introduces *tf.function*, which simplifies the applications and makes it very close to writing a normal Python application.

A sample *tf.function* definition looks like what's shown in the code snippet below. Here the *tf.function* declaration makes the user define a function as a TensorFlow operator, and all optimisation is applied automatically. Also, the function is faster than eager execution. APIs like *tf.control_dependencies*, *tf.global_variable_initializer*, and *tf.cond*, *tf.while_loop* are no longer needed with *tf.function*. The user defined functions are polymorphic by default, i.e., we may pass mixed type tensors.

```
test-pc:~$ cat tf-test.py
import tensorflow as tf

print(tf.__version__)
```